Digital Logic Design Final Project Report

Group members:

Nashra Azhar (Fa24b1-cs-081)

Zainab Khursheed (Fa24b1-cs-061)

Fizzah Rafiq (Fa24b1-cs-075)

Humna Sheraz (Fa24b1-cs-083)

IOT-Based Pet Feeder using ESP32 and web server

Introduction:

In today's busy lifestyle, managing timely feeding for pets becomes a challenge for many owners. To address this problem, we designed and implemented an **IOT-based Smart Pet Feeder** that allows users to remotely control the feeding process through **web server**. This system utilizes an **ESP32 microcontroller** and a built-in **web server** to automate and control a **servo motor**, which simulates the dispensing mechanism for feeding pets.

Project Overview

This project presents an IOT-enabled pet feeder that allows pet owners to feed their pets remotely through a web interface. The solution is built using an ESP32 development board, a servo motor, and jumper wires. When the user accesses the web server hosted on the ESP32 and clicks the "Feed Now" button, the servo motor activates, dispensing food. This system ensures convenience, especially for busy pet owners.

Objective:

- To create a smart and affordable IoT-based pet feeder using ESP32.
- To enable remote access and control via a local web interface hosted on the ESP32.
- To automate the pet feeding process, enhancing convenience for pet owners.
- To design a simple yet functional system using minimal hardware components.

Hardware Components Used

1. **ESP32 Development Board**
   - Acts as the main controller and Wi-Fi-enabled web server.
2. **Servo Motor (SG90 or equivalent)**
   - Rotates to dispense food when triggered.
3. **Jumper Wires**
   - Used to connect the servo motor to the ESP32 board.
4. **Micro USB Cable**
   - Used for programming and power supply.
5. **Wi-Fi Network**
   - Required to serve the web page and receive commands.

Connection Diagram

- **Servo Signal Pin**: GPIO 13 of ESP32
- **VCC**: Connect to 5V pin of ESP32
- **GND**: Connect to GND of ESP32

Functionality Description

1. **Wi-Fi Connection**: The ESP32 connects to a Wi-Fi network using the provided SSID and password.
2. **Web Server Setup**: A basic HTML interface is created on the root URL ("/") with a button labeled **"Feed Now"**.
3. **Feeding Process**:
   - ❖ On button click, the browser navigates to "/feed".
   - ❖ The ESP32 rotates the servo to 90 degrees to open the feeder.
   - ❖ After a 2-second delay, the servo returns to 0 degrees.
   - ❖ A confirmation message **"Pet Fed!"** is displayed.

Web Interface Preview

<h1>ESP32 Pet Feeder</h1>

<form action='/feed'>

  <button style='font-size:24px;'>Feed Now</button>

</form>

After feeding

<h2>Pet Fed!</h2>

Back

Complete Arduino Code

```
#include <WiFi.h>

#include <WebServer.h>

#include <ESP32Servo.h>  // Use the ESP32-compatible Servo library

// ✅ Wi-Fi credentials (your input)

const char* ssid = "muzammil 2.4 GHz";

const char* password = "MZ121121";

// Servo setup

Servo feederServo;

int servoPin = 13; // You can change to another PWM-capable pin

// Create web server on port 80

WebServer server(80);

// HTML interface
```

```
void handleRoot() {
  String html = "<h1>ESP32 Pet Feeder</h1><form action='/feed'><button style='font-size:24px;'>Feed Now</button></form>";
  server.send(200, "text/html", html);
}
// Feeding action
void handleFeed() {
  feederServo.write(90);  // Rotate servo to open
  delay(2000);            // Wait 2 seconds
  feederServo.write(0);   // Return to original
  server.send(200, "text/html", "<h2>Pet Fed!</h2><a href='/'>Back</a>");
}
void setup() {
  Serial.begin(115200);
 // Initialize servo
  feederServo.setPeriodHertz(50); // 50 Hz for standard servo
  feederServo.attach(servoPin, 500, 2400); // Min/max pulse width
  feederServo.write(0); // Start at 0 degrees
  // Connect to Wi-Fi
  WiFi.begin(ssid, password);
  Serial.print("Connecting to Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
```

```
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi connected!");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
  // Setup web routes
  server.on("/", handleRoot);
  server.on("/feed", handleFeed);
  server.begin();
  Serial.println("Web server started.");
}
void loop() {
  server.handleClient();
}
```

Applications

- Remote pet feeding (dogs, cats, rabbits, etc.)
- Useful for working professionals or travelers
- Expandable into a full smart pet care system

Future Scope

- Add real-time camera to monitor feeding
- Schedule auto-feeding times using RTC or timers
- Integration with cloud (e.g., Firebase, Blynk, or MQTT)
- Mobile app for global access (not just local network)

,

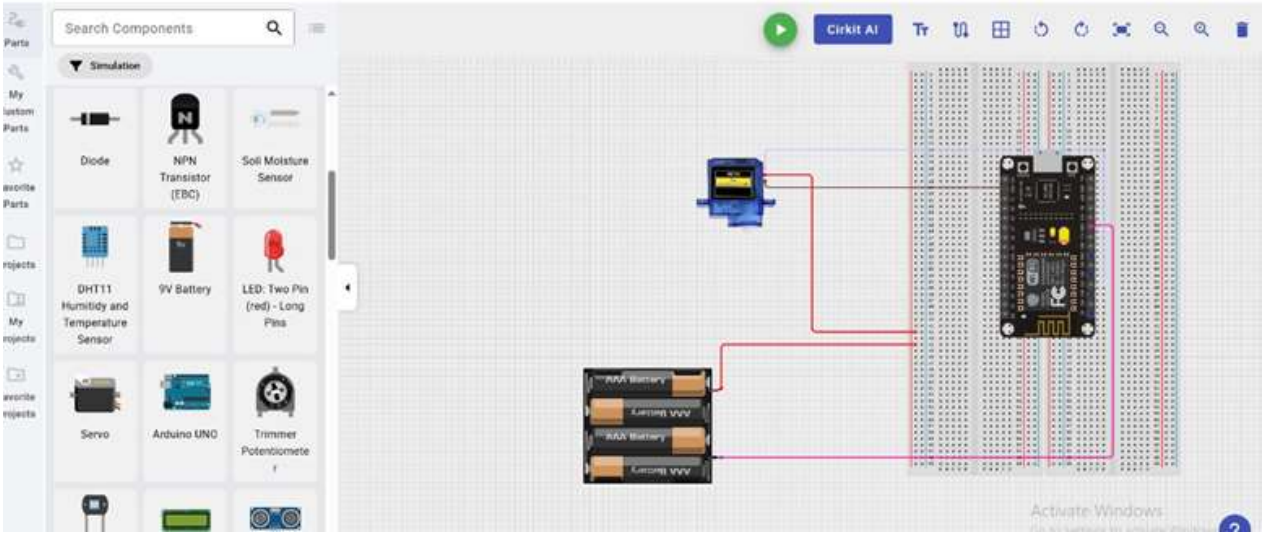
Conclusion

The IOT Pet Feeder using ESP32 is a simple yet effective project demonstrating how basic components and web technologies can solve real-world problems. With a clear user interface and reliable hardware integration, this project is a perfect example of smart automation tailored for pet care.

References

- ESP32 Documentation
- Arduino ESP32 Core
- ESP32Servo Library

Smart Pet Feeder:


Search Components
Simulation
Diode
NPN Transistor (EBC)
Soil Moisture Sensor
DHT11 Humitidy and Temperature Sensor
9V Battery
LED: Two Pin (red) - Long Pins
Servo
Arduino UNO
Trimmer Potentiometer
Cirkit AI
AAA Battery
AAA Battery
Activate Windows